

Git workflow and command evidence

A fully connected TensorFlow ANN classifies all 10 Fashion-MNIST classes. The workflow records code in Git and data/model artifacts in DVC. The final model achieves 88.43% test accuracy on 10,000 held-out examples; the required minimum is 85%.

Repository: <https://github.com/waheedsarem/fashion-ann-pipeline>

A1 A2 A8 Initialization and incremental history

Initialized main, committed README and credential/artifact ignores, then created dev. Separate commits implement parameters, prepare, preprocess, train and evaluate. Four git mv commands move scripts into src/; a later git rm removes the tracked scratch file. No submission commits are squashed.

```
$ git init -b main
$ git checkout -b dev
$ git mv preprocess.py src/preprocess.py
$ git rm scratch.txt
fa6a5f5 Remove obsolete scratch file with git rm
273b53b Reorganize independent pipeline scripts with git mv
```

A3 A4 Logs and differences

git log --oneline --graph --all shows branch topology compactly. git log --stat -3 adds changed-file and line counts for the last three commits. git log -p -1 displays the latest patch. git log main..dev selects commits reachable from dev but not main.

git diff shows working-tree versus index changes; git diff --staged shows index versus HEAD. Both captured the preprocessing comment. git diff main..dev compares the two branch tips. git diff main...dev compares their merge base to dev, isolating changes introduced on dev. Before rebasing, the two-dot diff also showed the README hotfix absent from dev; the three-dot diff did not.

A5 A6 Stash and rebase

```
$ git stash push -m "Mid-edit preprocessing documentation"
$ git stash list
stash@{0}: On dev: Mid-edit preprocessing documentation
$ git checkout main
$ git checkout dev
$ git stash pop
$ git rebase main
Successfully rebased and updated refs/heads/dev.
```

The stashed edit returned and was committed. A hotfix branch corrected README spelling and was merged into main. Rebasing replayed nine dev commits above that hotfix without conflict. Actual before/after graphs and every log/diff output are retained in evidence/part_a.txt.

A7 Reset on an isolated scratch branch

```
$ git reset --soft HEAD~1
$ git status --short
A  reset-demo.txt
$ git reset --hard HEAD~1
$ git status --short
(no tracked changes; reset-demo.txt no longer exists)
```

Soft reset moved HEAD while preserving the staged file. Hard reset restored both index and working tree. Evidence branches retain both throwaway commits for inspection.

Pipeline implementation and final configuration

B1-B4: prepare downloads the official 60,000/10,000 split. preprocess normalizes pixels to [0,1] and makes a stratified 48,000/12,000 train/validation split (seed 42). train uses Flatten -> Dense(ReLU) -> Dropout -> Dense(10, Softmax), Adam and sparse categorical cross-entropy. evaluate saves test loss, accuracy and a labeled confusion matrix.

The four scripts run independently from the repository root. Hyperparameters come only from params.yaml. models/model.h5 and models/history.csv record training. The test data is not used by model.fit. Fixed seeds and deterministic TensorFlow operations support repeatability within the recorded environment.

D1 D2 Complete final params.yaml and dvc.yaml

```
preprocess:
  test_size: 0.2
  seed: 42
train:
  seed: 42
  dense_units: 256
  dropout_rate: 0.2
  learning_rate: 0.001
  epochs: 20
  batch_size: 128

stages:
  prepare:
    cmd: python src/prepare.py
    deps:
      - src/prepare.py
    outs:
      - data/raw
  preprocess:
    cmd: python src/preprocess.py
    deps:
      - src/preprocess.py
      - data/raw
  params:
    - preprocess.test_size
    - preprocess.seed
  outs:
    - data/processed
  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed
    params:
      - train.seed
      - train.dense_units
      - train.dropout_rate
      - train.learning_rate
      - train.epochs
      - train.batch_size
    outs:
      - models
  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - models/model.h5
      - data/processed
    outs:
      - reports
    metrics:
      - metrics.json:
        cache: false
```

A3 A4 output excerpts (full logs in evidence/)

```
$ git log --oneline --graph --all
* f62cc99 Remove obsolete scratch file with git rm
* 615df0e Reorganize independent pipeline scripts
with git mv
* 1fe31cb Add temporary scratch file for
repository exercise

$ git log --stat -3
1 file changed, 1 deletion(-)
4 files changed, 0 insertions(+), 0 deletions(-)
1 file changed, 1 insertion(+)

$ git log -p -1
+++ /dev/null
@@ -1 +0,0 @@
-Temporary scratch file for the git rm
demonstration.

$ git log main..dev
commit f62cc99b61344d5b49d3f30...
Remove obsolete scratch file with git rm

$ git diff / git diff --staged
+# Validation splitting is stratified
+ to preserve class balance.
(excerpt; same added comment in both)

$ git diff main..dev
-# Fashion-MNIST ANN Pipeline
+# Fashion MNIST ANN Pipeline
$ git diff main..dev
(no README spelling change)
```

C5 to D2 migration: standalone .dvc pointers were created first and used for the conflict exercise. They were then removed as metadata only before dvc.yaml took ownership of the same paths. DVC does not allow duplicate output ownership. Cached objects and historical pointers remain available.

metrics.json is a DVC metric with cache: false and is committed to Git. An identical reports/metrics.json is cached alongside the confusion matrix. The current artifacts were uploaded and verified in Drive.

Measured results and dependency reproduction

D3 First complete dvc repro execution

```
Running stage 'prepare':
> python src/prepare.py
Generating lock file 'dvc.lock'
Updating lock file 'dvc.lock'
Running stage 'preprocess':
> python src/preprocess.py
Updating lock file 'dvc.lock'
Running stage 'train':
> python src/train.py
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
Updating lock file 'dvc.lock'
```

All four stages executed in sequence and generated dvc.lock. This run used 128 hidden units. The v1 tag preserves the earlier standalone baseline; pipeline-v1 records the first complete pipeline run after conflict resolution.

D4 D5 Change dense_units from 128 to 256

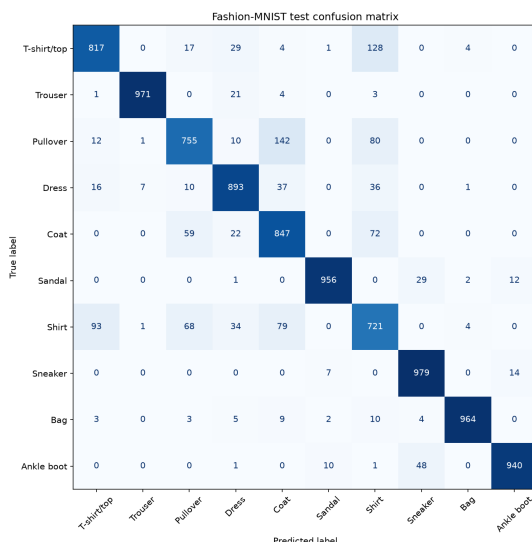
```
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train.py
Running stage 'evaluate':
> python src/evaluate.py
```

Only train depends on train.dense_units, so prepare and preprocess were skipped. Changing this parameter reran train, and the changed model reran evaluate. The updated params.yaml, dvc.yaml, dvc.lock and metrics.json were committed and tagged v2.

v1 and v2 test metrics

Version	Hidden units	Test accuracy	Test loss
v1	128	88.37%	0.337296
v2	256	88.43%	0.340086

Accuracy changed by +0.06 percentage points. Both evaluations use the same 10,000-image official test set. This is a measured comparison of two runs, not an estimate over repeated seeds.



Verification

Data shapes and pixel range checked.
ANN topology and epoch count checked.
Root and cached metrics agree.
Final dvc status is clean.
Final dvc repro executes no stages.

Full logs are in evidence/.
Environment: Python 3.12.14,
TensorFlow 2.20.0, DVC 3.63.0.

Conflict resolution and remote evidence

E1 E2 E3 Two contributors and two conflicts

After v1 was merged into main, teammate-sim changed preprocessing to float64 [0,1] values. Independently, main changed it to float32 [-1,1]. Each branch regenerated processed data, ran dvc add and committed its code and pointer. Merging produced simultaneous conflicts in src/preprocess.py and data/processed.dvc.

```
CONFLICT (content): Merge conflict in data/processed.dvc
CONFLICT (content): Merge conflict in src/preprocess.py
UU data/processed.dvc
UU src/preprocess.py
```

```
outs:
<<<<<<< HEAD
- md5: bcf5d68187d28f74ec911362d581f6e5.dir
  size: 47446101
=====
- md5: 587def07d8f546a8cc229ad88c0cad07.dir
  size: 50778235
>>>>>>> teammate-sim
  nfiles: 1
  hash: md5
  path: processed

<<<<<<< HEAD
return (images.astype(np.float32) / 127.5) - 1.0
=====
return images.astype(np.float64) / 255.0
>>>>>>> teammate-sim
```

E4 E5 Reconcile code and choose authoritative data

```
return (np.clip(images, 0, 255).astype(np.float64) / 255.0).astype(np.float32)
```

The resolution retains the required [0,1] range, performs division in float64 and stores float32. Regenerated data is authoritative. The complete uint8 mapping matches the baseline normalization. dvc add records the resolved hash and dvc checkout synchronizes the artifacts; dvc status reports a clean state before completing the merge.

```
$ dvc checkout
$ dvc status
Data and pipelines are up to date.
$ git commit -m "Resolve code and data conflicts ..."
$ dvc repro
Data and pipelines are up to date.
```

The final repro after the v2 merge skipped all stages. Merge markers, pointer hashes and resolution commands are retained in evidence/.

C1 C2 C3 C4 C5 Google Drive and submission status

DVC gdrive_storage points to the replacement folder. Google OAuth succeeded after adding the account as a tester to project sonorous-haven-510508-a0.

```
gdrive://1WCgR7ugUC_VjKzU-36cxLQN-WfbN2n-q
$ dvc status --cloud
Cache and remote are in sync.
Drive API listing: 21 folders, 20 files, 12 DVC hash objects; 178,413,292 bytes.
```

The v1, v2, teammate-sim and centered-normalization artifacts were pushed and verified against the Drive remote. A browser screenshot is unavailable; the folder contents were verified through the read-only Drive API.

GitHub: Public repository with full unsquashed history, branches and version tags.

References and evidence

References: tensorflow.org (Fashion-MNIST); dvc.org (Google Drive remote); git-scm.com (git diff). Captured command outputs accompany this report.